

# CSCI 12532

# Computer Architecture and Design

## Lecture 08

---

Ms. Meesha Mervyn  
Dept. of Computer Systems Engineering  
Faculty of Computing and Technology  
University of Kelaniya  
[meesham@kln.ac.lk](mailto:meesham@kln.ac.lk)

# Contents

1. What are I/O Modules?
2. System Bus
3. I/O Channels and Processors

# 1. WHAT ARE I/O MODULES?

Input/Output Modules, or I/O Modules, manage the communication between a CPU and a network, including the transfer of data, the management of power loads, and the control of machine functions.

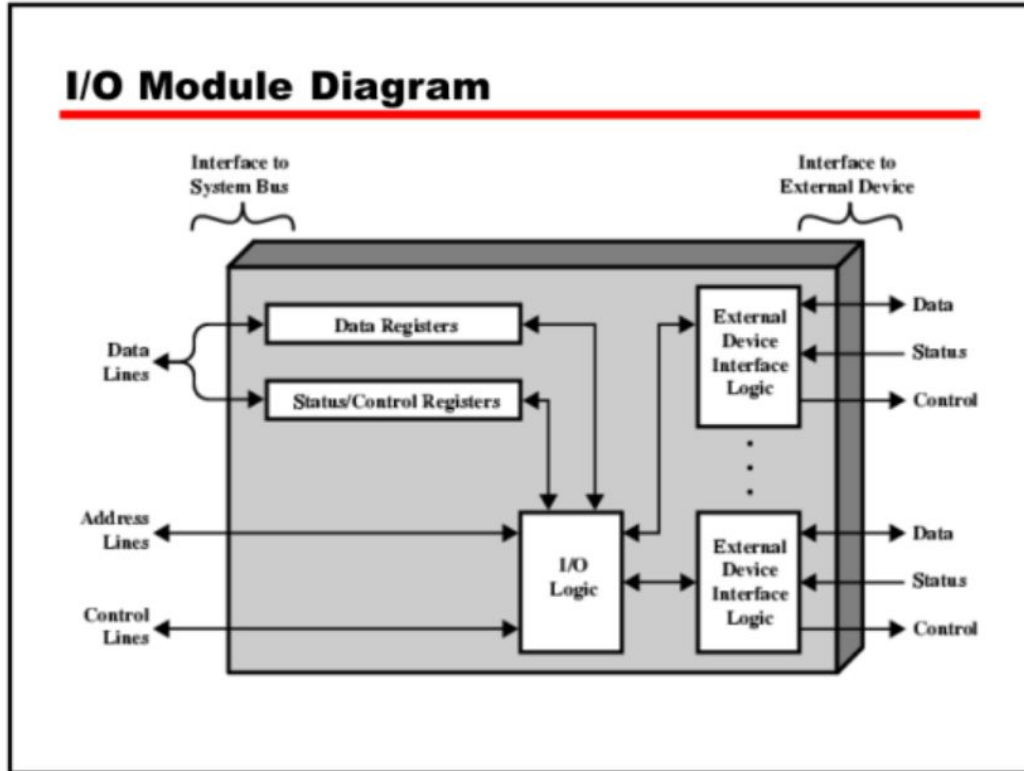
This enables system integrators to connect disparate devices, allowing greater control of the industrial network. I/O modules are especially helpful in instances where there exists legacy machinery, devices, and systems that are unable to natively communicate with a desired industrial protocol.

I/O modules help to extend a manufacturer's network to incorporate all manufacturing equipment, enabling greater control of the system as well as increased operational visibility. They also overcome the challenge of collecting peripheral data, which can come in various amounts, at different speeds, and in varying formats.

# Categories of Input/Output Operations

1. Sensory Input
  - Digital Input
  - Analog Input
2. Control Output
  - Direct Digital Output
  - Modulated Digital Output
  - Analog Output
3. Data Transfer
  - Parallel
  - Serial

# Structure of an I/O Module



# Key Components of I/O Module

## 1. System Bus Connection

Interfaces with the CPU/memory via signal lines (data, address, control buses).

## 2. Data Buffering

Data Registers Temporarily store incoming/outgoing data.

Ex: Holds keyboard input until the CPU is ready to process it.

## 3. Status & Control Registers

Status Registers: Report device state (e.g., READY, BUSY, ERROR).

Control Registers: Accept commands from the CPU (e.g., READ, WRITE).

Some registers may dual-function (status + control).

#### 4. Control Logic

Communicates with the CPU via control lines to:

- Receive commands (e.g., initiate a disk read).
- Send interrupts (e.g., "data ready").
- Handle bus arbitration (competing I/O requests).

#### 5. Address Recognition

Each I/O module has a unique address (or range, if controlling multiple devices).

Decodes addresses to select specific peripherals.

#### 6. Device-Specific Interface Logic

Custom circuitry to match electrical/protocol requirements of attached devices.

Examples:

UART (serial port logic).

SATA controller (for HDDs/SSDs).

# Functions of the I/O module

1. Control and timing
2. Processor communication
3. Device communication
4. Data buffering
5. Error detection

# Control and Timing Function

## **Why Control & Timing is Needed?**

The processor communicates with I/O devices unpredictably (depends on program needs). Main memory & system bus are shared resources between CPU and I/O operations.

Without proper coordination there could be,

- Bus contention (multiple devices trying to use the bus simultaneously).
- Memory access conflicts (CPU and I/O competing for memory bandwidth).
- Performance bottlenecks (slow I/O holding up CPU operations).

Ex: For Hard Drive Access, CPU sends a READ command → Then I/O module,

- Checks if the drive is ready.
- Requests bus access.
- Transfers data in chunks (buffering if needed).
- Verifies data integrity before passing it to CPU/memory.

## Key Responsibilities of I/O Module for Control & Timing

- Synchronization
  - Ensures CPU and peripherals operate at correct times (avoids data corruption).
  - Example: CPU waits for "ready" signal before reading from a slow device.
- Arbitration
  - Manages access to shared resources (bus, memory).
  - Techniques: Priority-based (high-speed devices get preference), Time-slicing (each device gets a turn).

- Buffering & Flow Control
  - Temporarily stores data to handle speed mismatches.
  - Example: A keyboard buffer holds keystrokes until the CPU processes them.
- Command Decoding & Execution
  - Interprets CPU commands (READ, WRITE, STATUS CHECK).
  - Translates them into device-specific signals.
- Error Handling
  - Detects transmission errors (parity checks, CRC).
  - Retries failed operations or notifies the CPU.

# Processor communication Function

## Key Steps in Processor-I/O Module Communication

### 1. Command Decoding

I/O module receives control signals from the processor via the control bus.

Commands vary by device type

Disk Drive: READ SECTOR, WRITE SECTOR, SEEK (move to a specific track)

SCAN (locate a sector ID)

Printer: PRINT CHAR, STATUS CHECK

### 2. Data Exchange

Data transfer happens over the data bus.

Direction depends on operation:

- Input (Read): Peripheral → I/O Module → Processor
- Output (Write): Processor → I/O Module → Peripheral

Buffering may be used to handle speed mismatches.

### 3. Status Reporting

I/O module provides feedback to the processor.

Common status signals:

- BUSY (module/peripheral is occupied)
- READY (data can be read/written)
- ERROR (e.g., disk read failure)

Example Scenario:

CPU sends READ command → I/O module replies "NOT READY" if still processing a previous request.

### 4. Address Recognition

Each I/O device has a unique address (similar to memory addresses).

I/O module decodes the address to select the correct peripheral.

Methods of addressing

- Memory-Mapped I/O (I/O devices share memory address space)
- Isolated I/O (dedicated I/O address space, using special instructions like IN/OUT in x86)

In summary, during processor communication

1. CPU sends command (via control bus).
2. I/O module decodes command and checks device status.
3. Data transfer occurs (if device is ready).
4. Status updates ensure synchronization.
5. Addressing ensures correct device selection.

# Device Communication Function

The I/O module facilitates interaction between the computer system and peripheral devices through three critical elements.

## 1. Commands

Control Signals: The I/O module sends device-specific commands (e.g., read, write, seek, reset).

Examples:

- Disk Drive: READ SECTOR, WRITE SECTOR, SEEK TRACK
- Printer: PRINT, FEED PAPER, STATUS CHECK
- Protocol Handling: Converts system-level commands into device-understandable signals.

## 2. Status Information

Device State Monitoring: Provides real-time feedback on peripheral status.

Common Status Signals:

- READY/BUSY: Indicates if the device can accept commands.
- ERROR: Reports malfunctions (e.g., paper jam in a printer).

Example Use Case: Before reading data, the CPU checks if the disk drive is READY or BUSY.

### 3. Data Transfer

#### Directional Flow

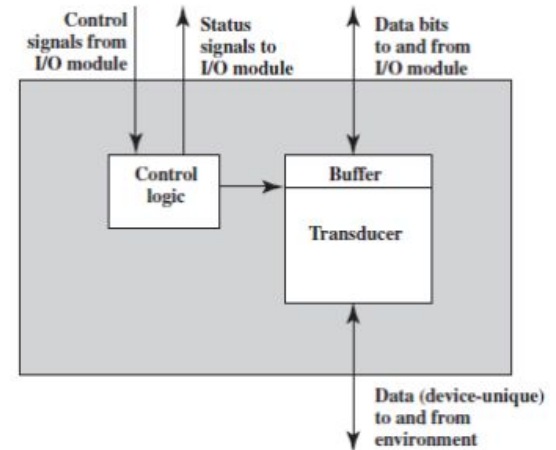
- Input Operations: Device → I/O Module → System (e.g., keyboard input).
- Output Operations: System → I/O Module → Device (e.g., display output).

Data Buffering: Temporarily stores data to manage speed mismatches (e.g., keyboard buffer).

Error Handling: Includes parity checks or CRC to ensure data integrity.

#### So why does Device Communication Matter?

- Ensures Synchronization: Coordinates timing between fast CPUs and slow peripherals.
- Enables Plug-and-Play: Standardized protocols (e.g., USB, SATA) allow seamless device integration.
- Improves Reliability: Status monitoring prevents data corruption during failures.



# Data Buffering Function

Resolves speed mismatches between:

- High-speed components (CPU, RAM)
- Low-speed peripherals (keyboards, printers, HDDs)
- Prevents bottlenecks in data transfer.

## How Buffering Works

### 1. CPU/Memory → Peripheral (Output Operation)

Data sent from RAM to I/O module in a high-speed burst.

I/O module stores data in its buffer.

Sends data to the device at a sustainable rate (e.g., printer speed).

## 2. Peripheral → CPU/Memory (Input Operation)

Data collected gradually from a slow device (e.g., keyboard input).

Buffered until a sufficient block is ready for efficient memory transfer.

Prevents memory bus congestion from slow transfers.

## 3. High-Speed Peripherals (e.g., SSDs, GPUs)

If a device is faster than memory bandwidth, the I/O module:

Buffers data to avoid overwhelming the system.

Coordinates burst transfers during memory idle cycles.

Ex:-

Print Spooling: Documents buffered before printing.

Network Cards: Packets buffered to handle variable transmission speeds.

# Error Detection Function

Ensures data integrity during transfers between CPU, memory, and peripherals. Identifies and reports hardware failures or data corruption.

## Types of Errors Detected

- Device-Level Errors (Mechanical/Electrical Failures)
- Data Transmission Errors (Bit Corruption)

## Error Handling Workflow

- Detection
- Reporting
- Recovery (if possible)

# I/O Module Structure & Functionality

System Bus Connection Links to CPU/memory via signal lines (data, address, control buses).

- Data Handling  
Data Registers Buffer incoming/outgoing data to manage speed mismatches.
- Control & Status  
Status Registers Report device state (e.g., READY, BUSY, ERROR).  
Control Registers Accept high-level commands from CPU (e.g., READ, WRITE).
- Processor Interaction  
Control Lines - CPU uses these to send commands (e.g., initiate transfer).  
I/O module uses them for Arbitration (competing bus requests) & Status signals (e.g., interrupt requests).

- Address Management  
Each I/O module:
  - Recognizes unique addresses for its peripherals.
  - Generates device-specific signals during data transfers.
- Device-Specific Logic  
Custom circuitry to handle Protocol translation (e.g., USB to serial) & Electrical interfacing (e.g., voltage conversion).

### **Simplified view of peripherals for the processor**

Acts as an I/O Processor:

- Presents a high-level interface (hides hardware complexity).
- Offloads low-level tasks from CPU (e.g., error checking, timing).

Ex: DMA controllers, GPU command processors.

## 2. SYSTEM BUS

A system bus, in its essence, is a set of parallel wires or traces on a circuit board that facilitates communication between the central processing unit (CPU), memory, and peripheral devices. It is the backbone of a computer's internal communication network, responsible for transferring data, addresses, and control signals. The system bus is subdivided into three primary buses: the data bus, the address bus, and the control bus, each serving a distinct function in the communication process.

## How It Works?

Step 1: CPU places an address on the address bus (e.g., to fetch data from RAM).

Step 2: Control bus signals read/write and synchronizes the transaction.

Step 3: Data is transferred via the data bus (e.g., RAM → CPU for a read operation)

# Basic functions of a system bus

- Internal Function (Memory Bus)

Connects internal hardware components (CPU, RAM, cache).

Facilitates high-speed communication within the computer.

Example: CPU fetching instructions from RAM.

- External Function (Expansion Bus)

Links external devices (printers, mice, USB drives) to the system.

Uses standardized interfaces (USB, PCIe, Thunderbolt).

Example: Connecting a wireless printer via USB.

- Data Sharing

Transfers data in parallel across multiple lines.

Bus width (e.g., 32-bit, 64-bit) determines data capacity per transfer.

Larger systems (e.g., servers) have wider/faster buses for higher throughput.

- Addressing

Specifies memory/I/O locations for data transfer.

CPU sends addresses to read/write from specific memory cells or devices.

Example: CPU requests data from RAM at address 0xFFFF0000.

- Power Distribution

Supplies electrical power to connected devices (e.g., USB peripherals).

Ensures stable voltage for signal integrity and timing.

# Types of system bus

## 1. Data Bus

Transfers actual data between components (CPU ↔ Memory ↔ I/O Devices).

Characteristics

- Bidirectional (can send and receive data).
- Width (e.g., 32-bit, 64-bit) determines how much data is transferred at once.

Example: CPU fetching an instruction from RAM.

## 2. Address Bus

Carries memory/I/O addresses to specify where data is read from or written to.

Characteristics

- Unidirectional (CPU → Memory/I/O).
- Width determines maximum addressable memory (e.g., 32-bit = 4GB limit).

Example: CPU sending address 0xA0000000 to read data from RAM.

### 3. Control Bus

Manages timing, coordination, and commands for bus operations.

Key Signals:

- Read/Write (determines data flow direction).
- Clock (synchronizes operations).
- Interrupts (alerts CPU to urgent events).
- Bus Request/Grant (for DMA or multi-master access).

Purpose: Ensures accurate, secure, and synchronized data transfers.

In Summary,

Data Bus: The "highway" for data.

Address Bus: The "GPS" for locating data.

Control Bus: The "traffic controller" for coordination.

# System bus implementation

## 1. Synchronous Bus

This is Clock-Driven. Operates in sync with the computer's clock signal, and has Fixed Timing. All operations (data/address/control) follow a strict clock cycle schedule.

Ex: Data is read/written only at the rising edge of the clock.

Advantages:

- High speed (optimized for predictable transfers).
- Simple design (no handshaking needed).

Disadvantages:

- Inflexible for slow devices (must match bus speed).
- Prone to timing skew in long buses.

Use Cases:

- CPU-RAM communication (e.g., DDR SDRAM).
- High-speed internal buses (e.g., Front-Side Bus in older PCs).

## 2. Asynchronous Bus

This is Clock-Independent: No central clock; devices use handshake signals to coordinate.

Flexible Timing: Request/Acknowledge protocol (e.g., READY and STROBE signals).

Ex: Printer asserts READY before accepting data.

Advantages:

- Compatible with mixed-speed devices (e.g., USB, legacy ports).
- Robust over long distances (immune to clock skew).

Disadvantages:

- Slower due to handshaking overhead.
- Complex logic for error handling.

Use Cases:

- External peripherals (e.g., USB, SCSI).
- Legacy systems (e.g., ISA bus).

# 3. I/O Channels and Processors

## Evolution of the I/O Function & I/O Channels

As computer systems have evolved, there has been a pattern of increasing complexity and sophistication of individual components. Nowhere is this more evident than in the I/O function.

### Step 1 – Direct CPU Control

- CPU directly controls a peripheral device
- Example: Simple microprocessor-controlled devices (e.g., a basic thermostat)

## Step 2 – Controller/Module Added (Programmed I/O)

- An I/O module (controller) is added
- CPU uses programmed I/O (no interrupts)
- CPU becomes less involved in device-specific details

## Step 3 – Interrupts Added

- Same hardware as Step 2, but with interrupts
- CPU no longer waits for I/O operations → increased efficiency

## Step 4 – Direct Memory Access (DMA)

- I/O module gets direct access to memory via DMA
- Transfers whole blocks of data to/from memory without CPU intervention
- CPU involved only at start and end of transfer

## Step 5 – I/O Channel (Specialized Processor)

- I/O module becomes an I/O channel – a processor with I/O-specific instructions
- CPU directs it to execute an I/O program stored in memory
- I/O channel fetches/executes instructions without CPU help
- CPU is interrupted only after the entire sequence of I/O activities is finished

## Step 6 – I/O Processor (with Local Memory)

- I/O module has its own local memory → a full computer
- Controls a large set of I/O devices with minimal CPU involvement
- Common use: Controlling many interactive terminals (e.g., terminal concentrators)

# What Is an I/O Channel?

- Can execute I/O instructions, giving full control over I/O operations
- CPU does not execute I/O instructions – they reside in memory for the channel
- CPU initiates transfer by telling channel to run a program in memory
- The program specifies: devices, memory areas, priority, error handling

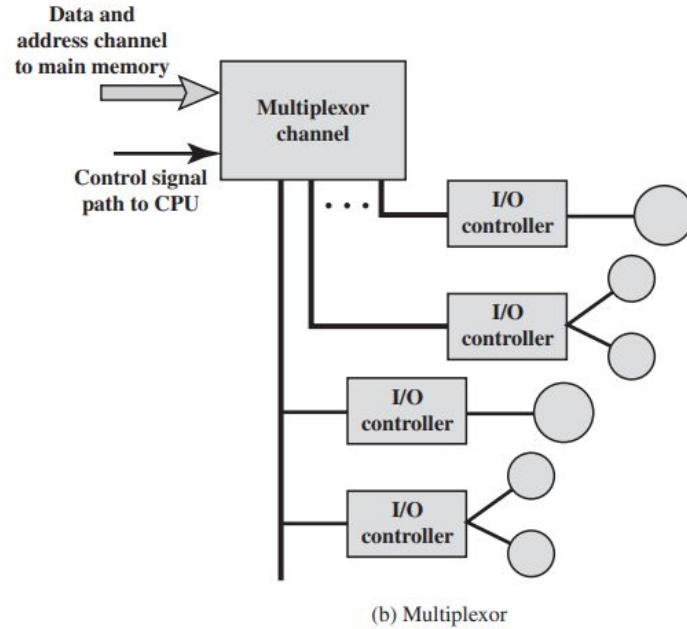
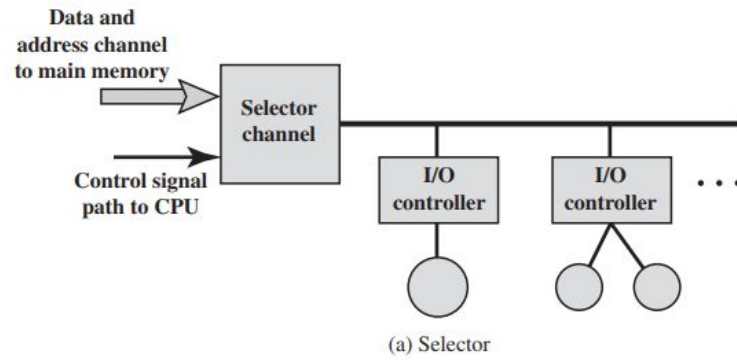
# Two Types of I/O Channels

## 1. Selector Channel

- Controls multiple high-speed devices
- Dedicated to one device at a time (selects one device for data transfer)
- Works with device controllers (I/O modules)

## 2. Multiplexor Channel

- Handles I/O with multiple devices at the same time
- Subtypes:
  - Byte multiplexor – for low-speed devices; interleaves characters
  - Example: Three devices with streams A1A2A3..., B1B2..., C1C2... → A1 B1 C1 A2 C2 A3 B2 C3 ...
  - Block multiplexor – for high-speed devices; interleaves blocks of data



# Input-Output Processor (IOP)

A specialized processor that manages I/O operations independently of the CPU.

Key feature: Direct Memory Access (DMA) capability (transfers data directly to/from memory).

## How It Works?

- System Architecture  
Computer splits into; Memory unit (shared by CPU and IOPs) & Multiple processors (CPU for computation, IOPs for I/O tasks).
- Task Handling  
IOP controls all I/O devices (disks, printers, etc.).  
CPU delegates I/O tasks via channel programs (stored in memory).
- Autonomy  
Fetches and executes its own I/O-specific instructions.  
Examples: Disk seek, printer spooling, network packet handling.

# Comparison with CPU

| Feature       | CPU                   | IOP                               |
|---------------|-----------------------|-----------------------------------|
| Primary Role  | General computation   | Dedicated I/O management          |
| Memory Access | Standard read/write   | DMA-enabled (bypasses CPU)        |
| Instructions  | Full instruction set  | I/O-focused (e.g., SCSI commands) |
| Dependency    | Manages entire system | Works under CPU supervision       |

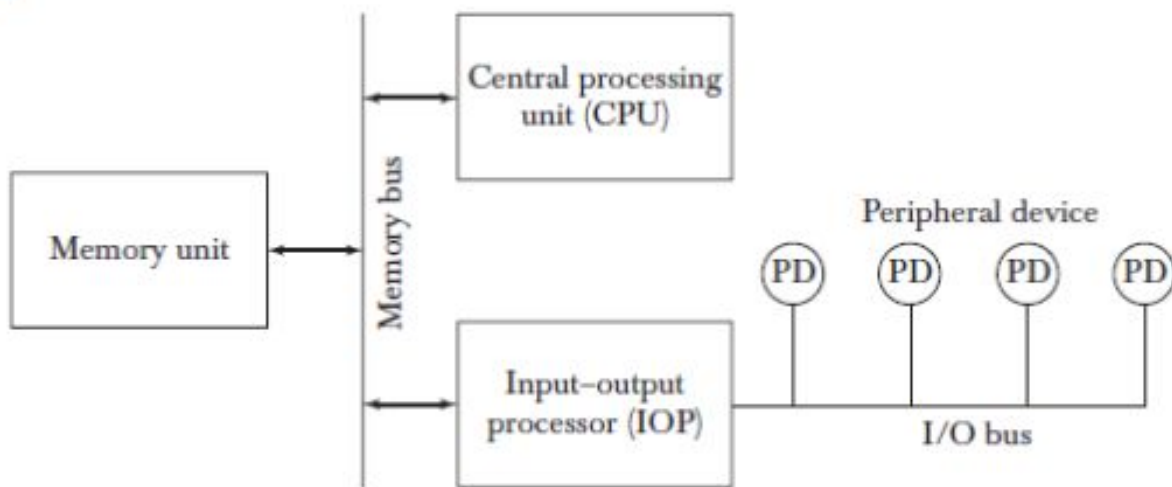


Fig.05: A System configuration with IOP

# Characteristics of an I/O Processor (IOP)

- Specialized Instruction Set

I/O-Focused Commands executes device-specific operations (not general-purpose like a CPU).

Examples:

- ❖ Disk: READ SECTOR, WRITE SECTOR, SEEK, FORMAT, ERROR CHECK.
- ❖ Tape: REWIND, FORWARD, BOT (Beginning of Tape), EOT (End of Tape).
- ❖ Printer: LINE FEED, FORM FEED, PRINT, STATUS CHECK.

- I/O Program Execution

Can run autonomous I/O programs (stored in memory).

Ex: A disk IOP executes a sequence of commands (seek → read → verify).

CPU offloading: CPU initiates the program, then the IOP handles execution.

- Multi-Device Support

Decodes commands for diverse peripherals (disks, tapes, printers, NICs).

Each device type has a unique command set (e.g., SCSI vs. USB protocols).

- High-Speed Data Transfer

DMA (Direct Memory Access) - Transfers data directly to/from memory without CPU intervention.

Enables burst transfers (e.g., streaming data from SSD to RAM).

Buffering: Uses onboard registers to smooth speed mismatches.

- Scalability & Flexibility

Supports multiple devices simultaneously (via channels or multiplexing).

Hot-swappable in modern systems (e.g., USB, Thunderbolt)

# Functioning of IOP

## 1. CPU Initiates I/O Task

- Provides the memory address of the I/O program (channel program) to the IOP.
- Ex: CPU says, "Run the disk-read program at memory location 0x5000."

## 2. IOP Takes Over

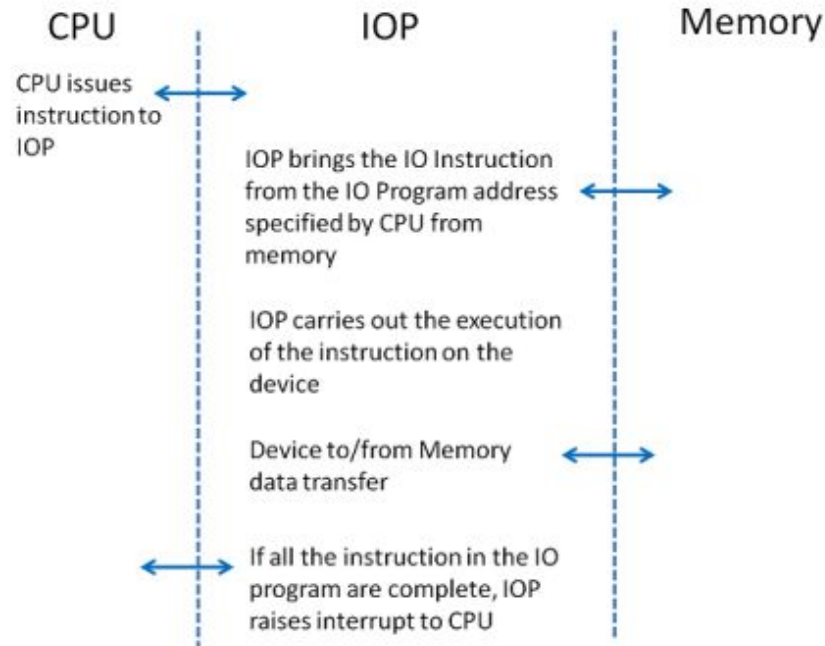
- Fetches the I/O program from memory.
- Decodes & Executes device-specific commands (e.g., SEEK, READ).
- Manages Data Transfer autonomously using DMA (no CPU involvement).

## 3. Completion & Interrupt

- After transfer, IOP raises an interrupt to notify the CPU.
- CPU handles post-processing (e.g., checks status, triggers next task).

IOP acts as a "co-pilot" for I/O

- CPU delegates (gives program address).
- IOP executes (DMA + device commands).
- IOP alerts (interrupt on completion).



# CPU-IOP Instruction Format

## 1. CPU Opcode (Operation Code)

Identifies the instruction as an I/O command (not a CPU computation).

Example:

0x4A (hypothetical opcode for "execute I/O program").

Tells the CPU: "This is an instruction for the IOP, not for you."

## 2. I/O Device Address

Specifies which peripheral device the I/O program targets.

Implementation:

Unique address per device (e.g., 0x01 = disk, 0x02 = printer).

May include sub-addresses (e.g., 0x01:0xFE = disk sector 254).

Example:

CPU sends: "Run the program on Device 0x03 (tape drive)."

### 3. I/O Program Address (Channel Program Address)

Points to the memory location of the I/O program (sequence of commands).

How It Works?

- Stored in main memory (e.g., at address 0x5000).
- Contains device-specific ops (e.g., SEEK, READ, STATUS CHECK).

Example:

Address 0x5000 holds:

[SEEK TRACK 12]

[READ SECTOR 5]

[VERIFY CHECKSUM]



# I/O Program

Similar to a regular program, but contains device-specific I/O instructions. Stored in main memory; fetched/executed by the IOP (not CPU).

## Key Components of an I/O Instruction

### 1. I/O Opcode

Specifies the operation to perform (e.g., READ, WRITE, SEEK).

Example:

- 0x01 = READ\_SECTOR (disk).
- 0x02 = PRINT\_LINE (printer).

## 2. Data Transfer Instructions

Memory Address: Source/destination of data in main memory (e.g., 0xA0000000).

Accessed via system bus (IOP competes for bus access using arbitration).

Data Transfer Count: Size of data to transfer (e.g., 512 bytes for a disk sector).

## 3. Control Information

Error Handling: Bits to enable retry on failure or interrupt on error.

Interrupt Settings: Whether to notify CPU after completion (or only on errors).

Chaining:

"Chain" bit indicates if another I/O instruction follows (for multi-step ops).

"End" bit marks the last instruction in the program.

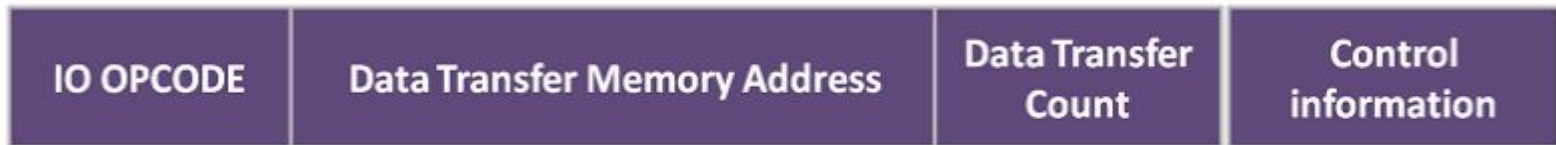
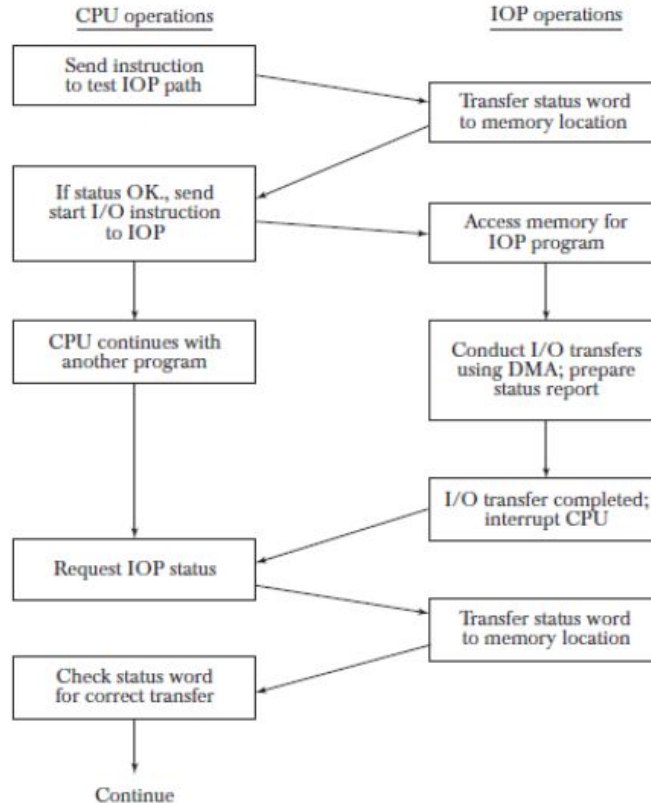


Fig.09: IOP Instruction from Memory

# CPU—IOP Communication



# Advantages of an I/O Processor (IOP)

## 1. Offloads CPU from I/O Tasks

CPU Delegation: The CPU only needs to initiate I/O operations (by providing the I/O program address).

Parallel Processing: CPU can continue executing computations while the IOP handles data transfers.

Example: A server processes user requests while the IOP manages disk reads/writes.

## 2. High-Speed Data Transfer (DMA Support)

Direct Memory Access (DMA): IOP transfers data directly between peripherals and memory without CPU intervention.

Eliminates per-byte CPU overhead (faster than programmed I/O).

Burst Mode Transfers: Moves large blocks of data efficiently (e.g., loading game textures).

## 3. Supports Complex I/O Operations

Multi-Step Commands: Executes chained I/O programs (e.g., SEEK → READ → VERIFY).

Device-Specific Handling: Dedicated logic for disks, printers, networks, etc.

#### 4. Reduces System Bottlenecks

Bus Arbitration: IOP manages contention for the system bus (no CPU involvement).

Buffering: Smooths speed mismatches between fast CPUs and slow devices (e.g., keyboards).

#### 5. Scalability for Multiple Devices

Concurrent I/O: One IOP can manage multiple peripherals (disks, NICs, GPUs).

Hot-Swappable: Modern IOPs (e.g., USB controllers) support plug-and-play devices.

#### 6. Error Handling & Reliability

Automatic Retries: Retransmits data on failure (e.g., disk read errors).

Status Reporting: Alerts CPU only on critical events (reduces interrupt storms).

# Thank You

09.06.2026